

Faculty of Engineering and Architectural Science

Department of Mechanical, Industrial, and Mechatronics Engineering

Course Number	COE718
Course Title	Embedded Systems Design
Semester/Year	Fall 2025
Instructor	Gul N. khan
Teaching Assistant	

Lab Report No.	1
-----------------------	----------

Report Title	Introduction to uVision and ARM Cortex M3
--------------	--

Section No.	05
Due Date of Lab Performance	Tuesday Sept 16, 2025 at 6 PM
Report Submission Date	

Name	Student ID	Signature*
Hamza Malik	501112545	H.M

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/policies/pol60.pdf>.*

Introduction

This report outlines the procedures and outcomes of the first lab exercise for **COE718**, which serves as an introduction to the Keil uVision integrated development environment (IDE) and the ARM Cortex M3 microcontroller. The primary objective was to develop a C program that interfaces with the joystick on a Keil MCB1700 development board. The program is designed to detect the direction of joystick movement and provide visual feedback through the board's LEDs and its graphical LCD. This exercise provides foundational knowledge in embedded systems programming, including project setup, hardware interfacing, and debugging.

Procedure

The lab was conducted following the detailed steps provided in the lab manual. The process began with the setup of a new project in Keil uVision. The project was configured for the NXP LPC1768 microcontroller, which is the core of the MCB1700 board. Necessary software packs were installed, and the Blinky project was used as a starting template. The project was then modified to incorporate the required joystick functionality. This involved adding the provided KBD.c and KBD.h files, which contain the necessary functions for initializing and reading the joystick's state. The main application file, Blinky.c, was then edited to continuously poll the joystick's status and update the LEDs and LCD accordingly. The software was compiled and debugged using the simulation tools within Keil uVision before being flashed onto the MCB1700 board. The final step was to test the program on the physical hardware to verify its functionality.

Project Workspace in Keil uVision

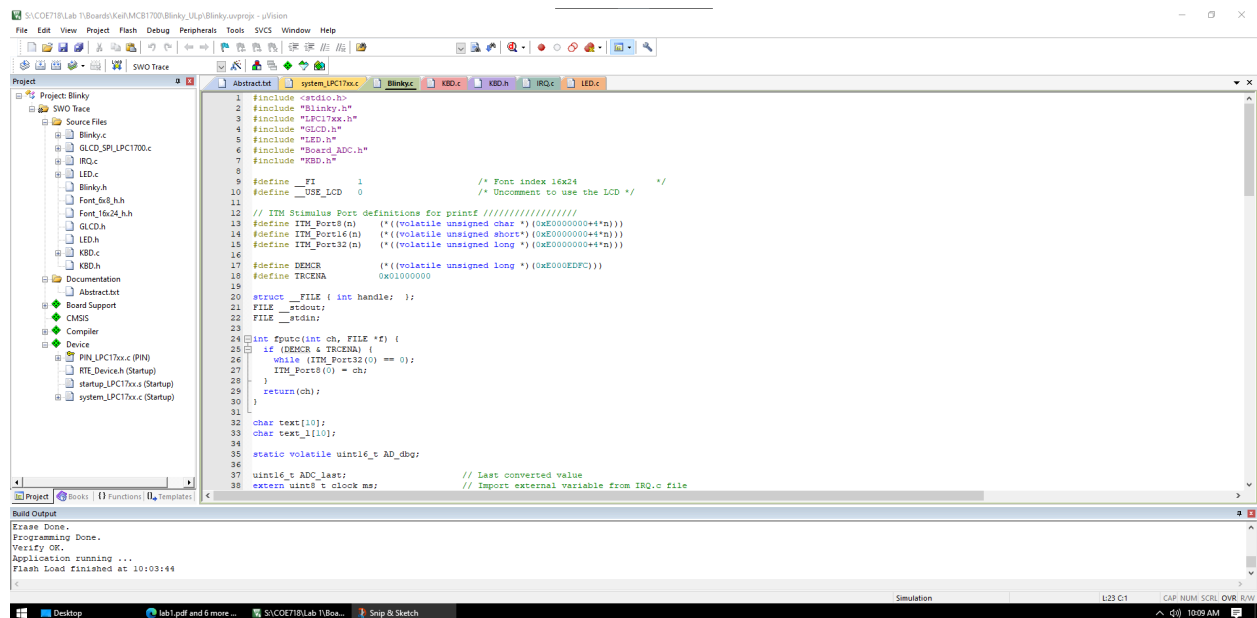


Figure 1: Blinky.c - Setup and Declarations

Figure 1 displays the complete project structure for Lab 1 within the Keil uVision IDE. It shows all the essential source and header files that work together, including the main application file Blinky.c, the joystick driver KBD.c, and various peripheral drivers.

The initial section of the Blinky.c file acts as the program's core setup area, performing several critical jobs:

- **Import Libraries:** It includes all the necessary code for the microcontroller (LPC17xx.h) and its peripherals (LCD, LED, etc.).
- **Enable Debugging:** It configures the system to allow printf statements to send debug messages to the computer for testing.
- **Declare Global Variables:** It creates variables, like AD_dbg for the ADC value, that need to be accessible throughout the entire program.

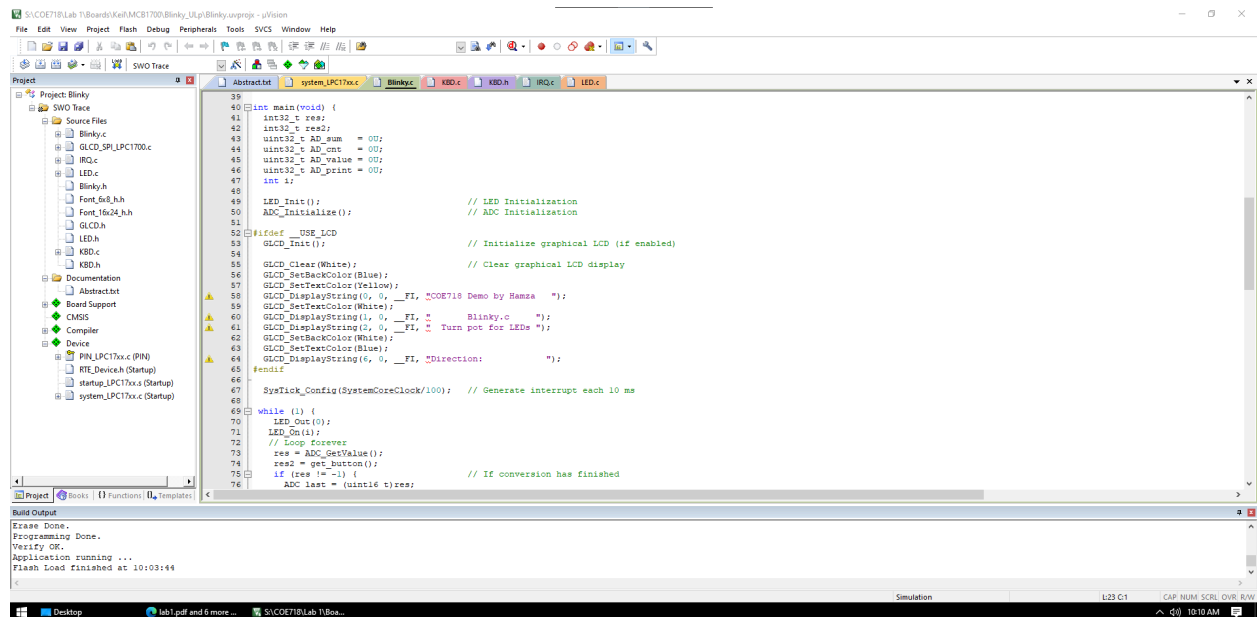


Figure 1.1: Blinky.c - Hardware Initialization

Figure 1.1 displays the start of the main function and is focused on getting the hardware ready.

- **Initialize Peripherals:** It calls functions to prepare the LEDs and the Analog-to-Digital Converter for use.
- **Draw UI on LCD:** It initializes the LCD screen, clears it, and prints the static text like the title and instructions.
- **Start the "Heartbeat":** It configures the system's main timer (SysTick) to fire at regular intervals (every 10ms), providing a consistent time base for the application.

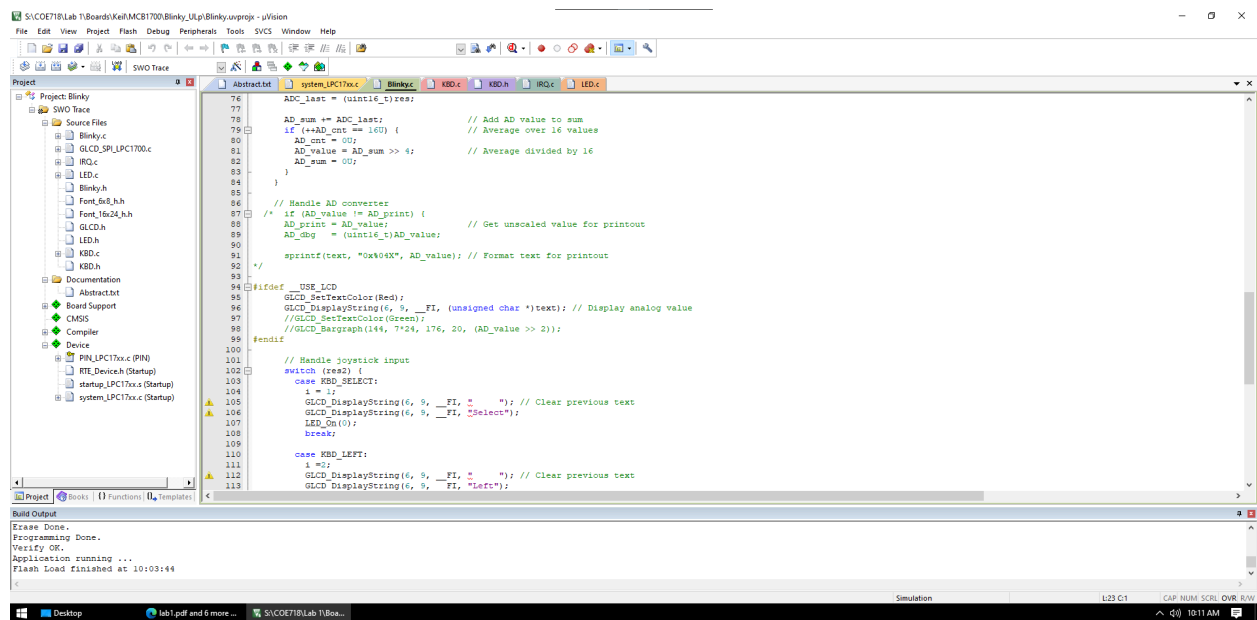


Figure 1.2: Blinky.c - Main Loop and Input Processing

Figure 1.2 displays the core operational logic of the program.

- **Infinite Loop:** The while(1) ensures the code runs continuously.
- **Read Inputs:** Inside the loop, it reads the current value from the potentiometer (ADC_GetValue) and, most importantly, from the joystick (get_button).
- **Decision Hub:** It uses a switch statement to check which direction the joystick was pressed. The code shown for the "Select" case is the template for all responses: update the LCD and turn on an LED.

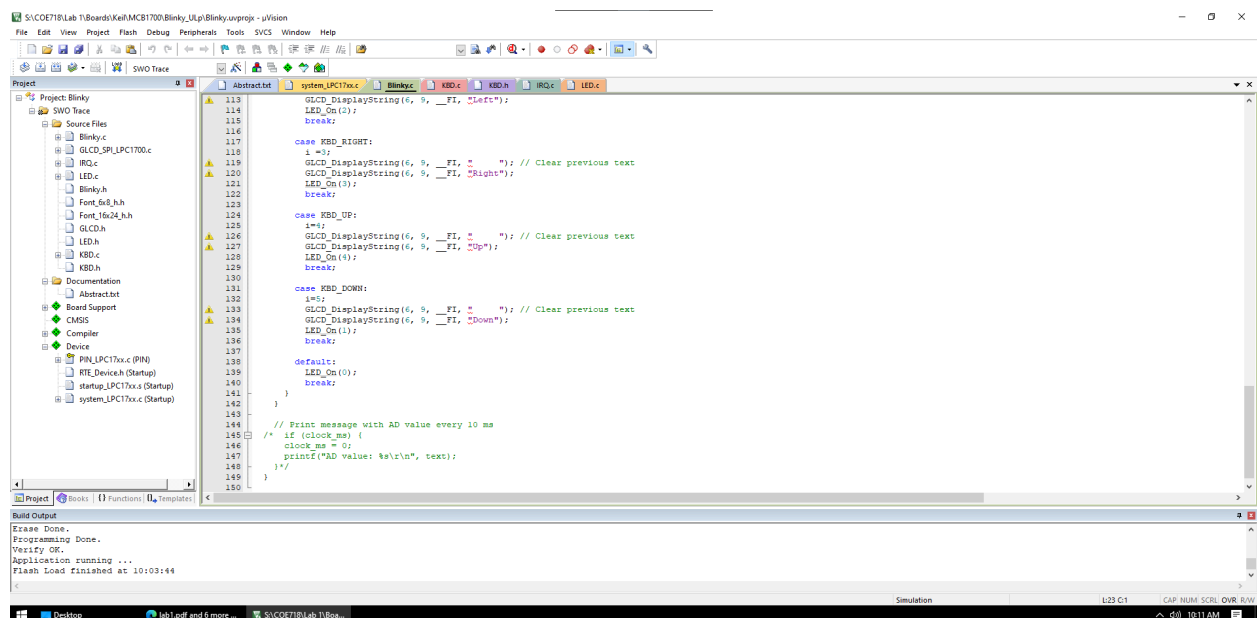


Figure 1.3: Blinky.c - Joystick Direction Logic

Figure 1.3 displays the rest of the program's decision-making logic.

- Handle All Directions: It contains the specific code blocks for each joystick direction (Up, Down, Left, Right). Each one follows the same pattern: clear the old text on the LCD, display the new direction, and light up a unique LED.
- Handle No Input: The default case at the end runs when no joystick button is pressed. Its purpose is to turn off the directional LEDs.

Use Arrow Up and Arrow Down to select a turn, Enter to jump to it, and Escape to return to the chat.

Hardware Demonstration

Figure 2 captures the program running on the MCB1700 development board, demonstrating the successful implementation of the lab's objectives. It showcases the final, functional state of the project. The LCD screen is actively displaying the custom user interface, including the titles "COE718 Demo by Hamza" and "Blinky.c". Crucially, it is showing Directions: Select, Up, Down, Left, Right, which confirms that the program is correctly reading the joystick's state in real-time. Simultaneously, a red LED on the board is illuminated. This provides a secondary, hardware-level confirmation that a direction has been detected and the corresponding output pin has been activated as programmed. The setup demonstrates a complete input-processing-output cycle, fulfilling the core requirements of the assignment.

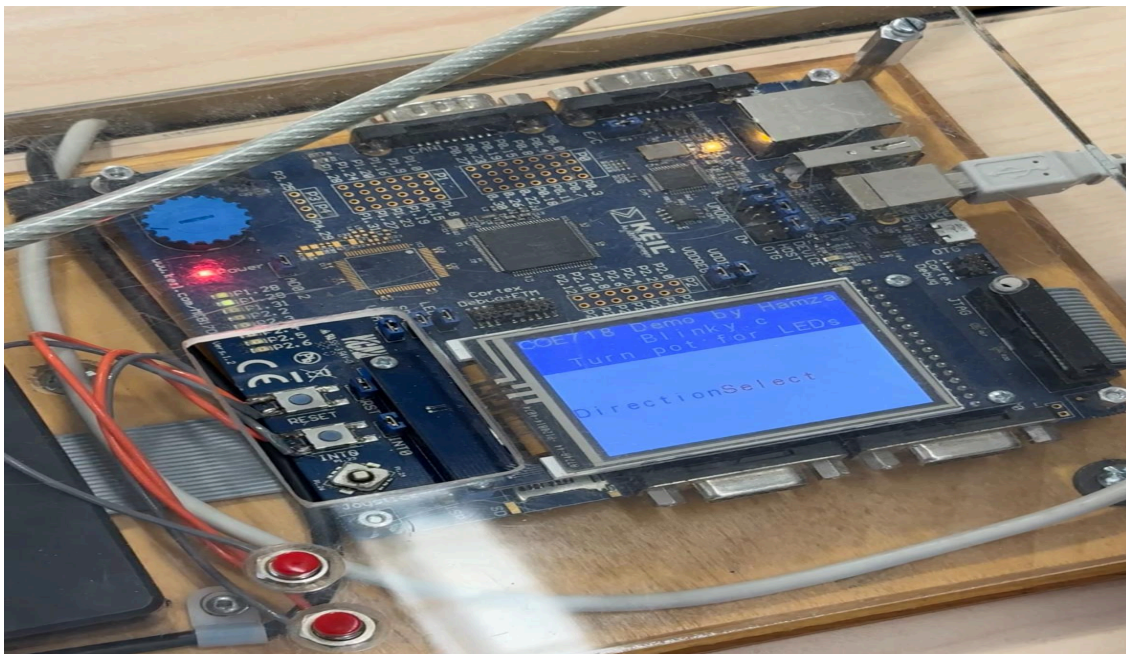


Figure 2: MCB1700 Development Board Direction Select

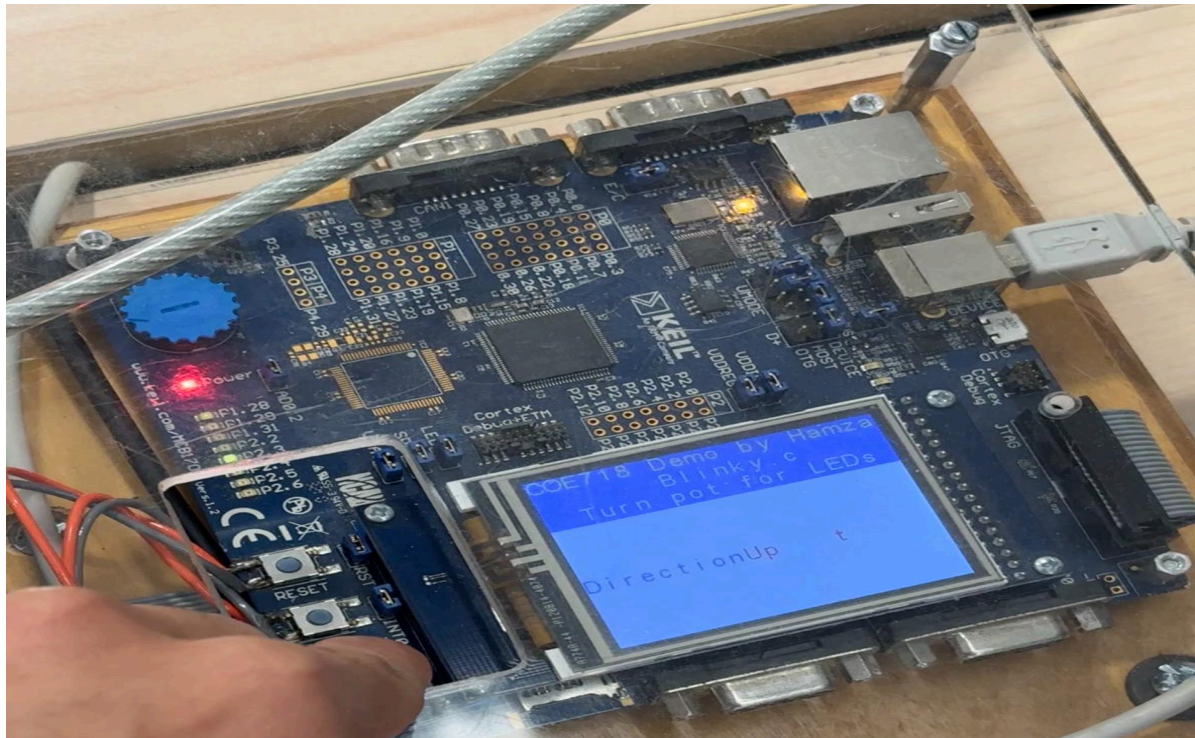


Figure 2.1: MCB1700 Development Board Direction Up

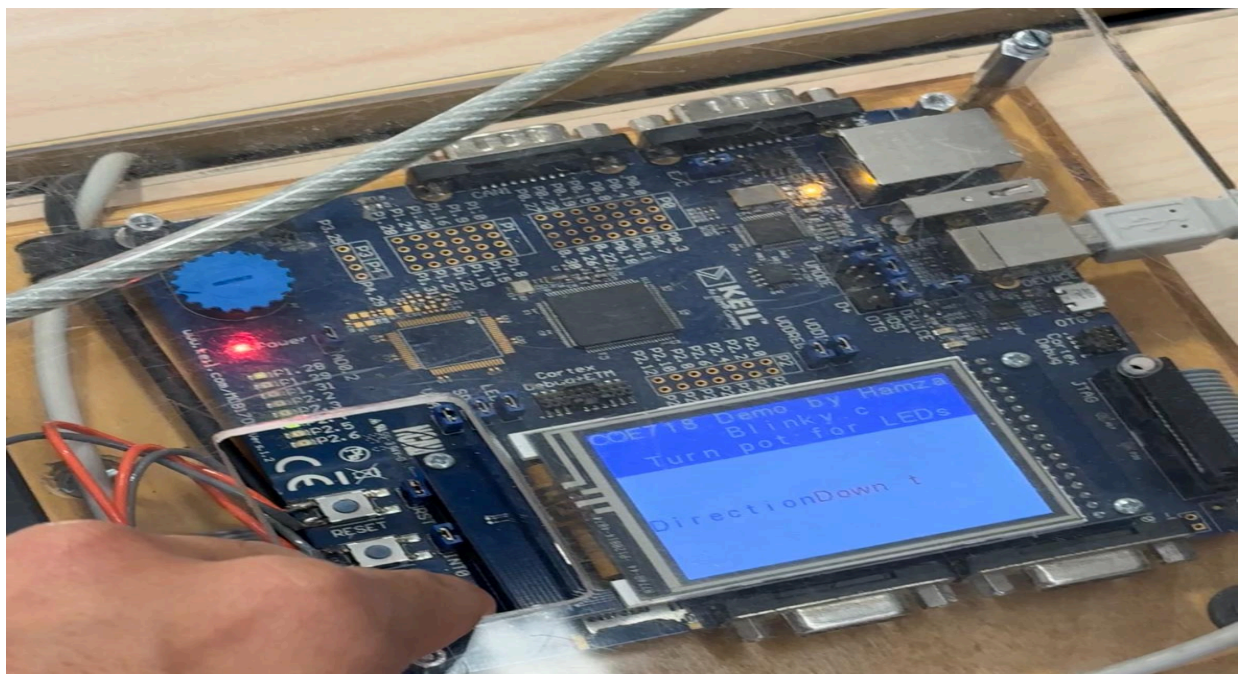


Figure 2.2: MCB1700 Development Board Direction Down

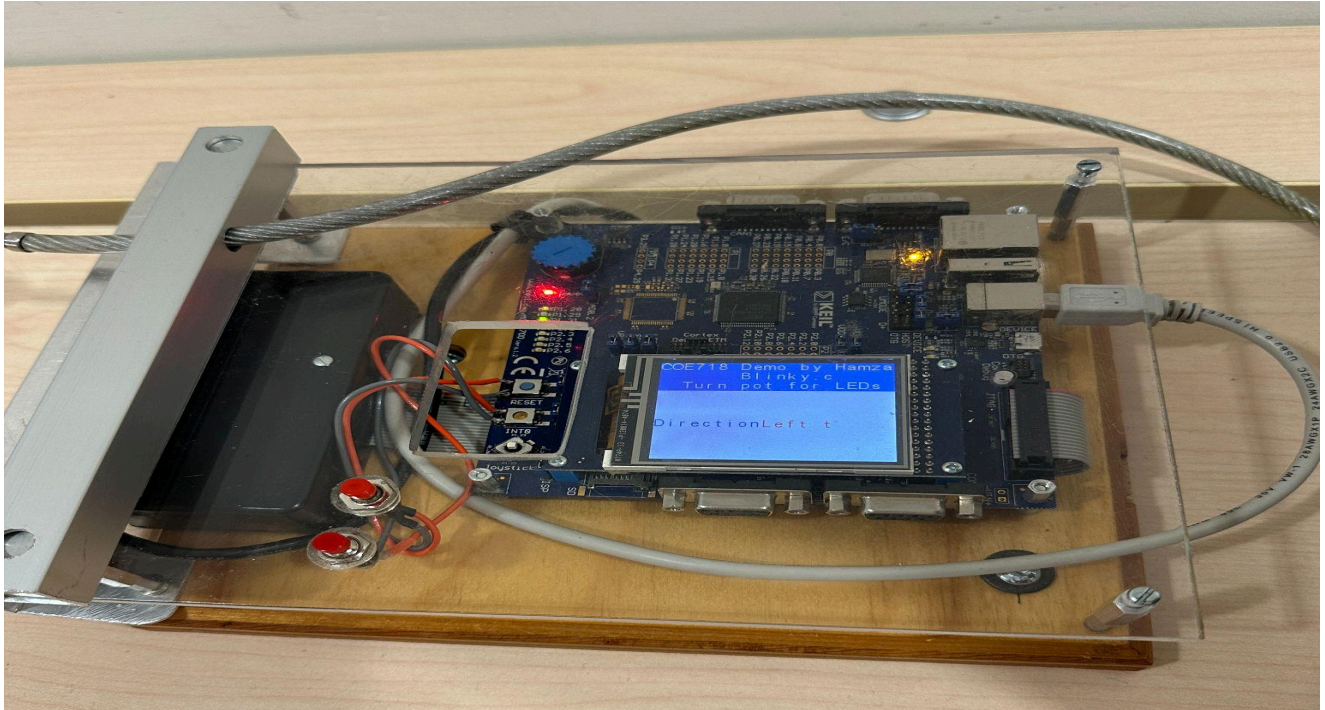


Figure 2.3: MCB1700 Development Board Direction Left

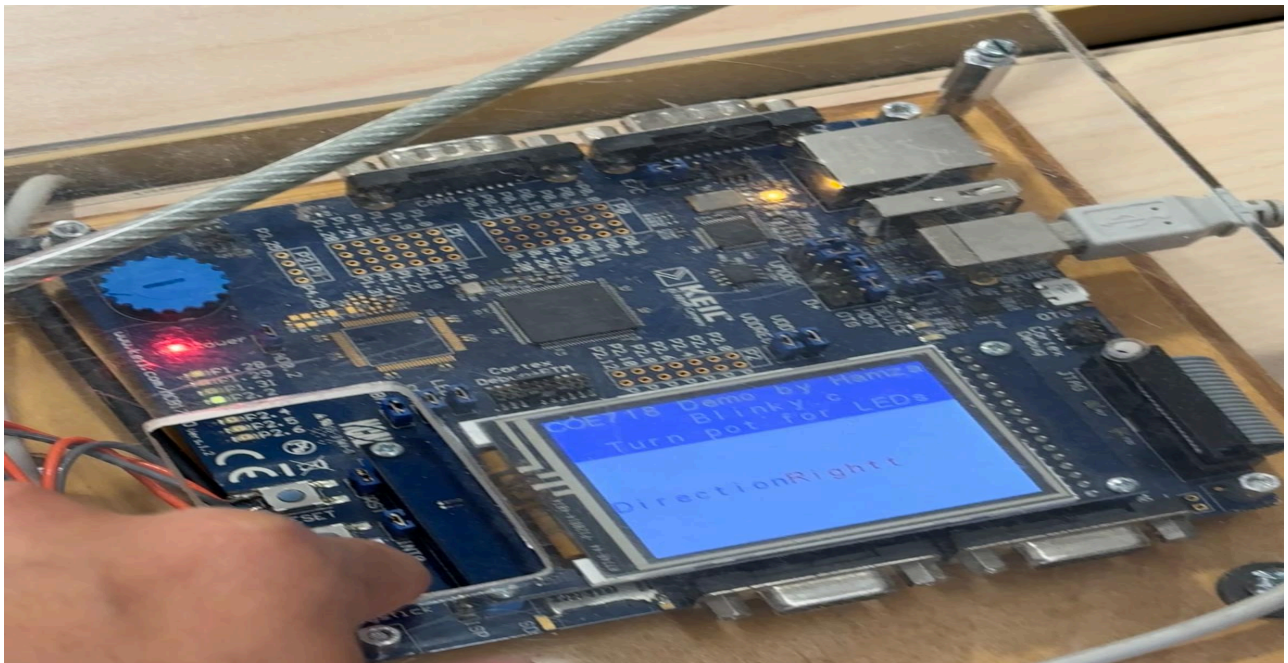


Figure 2: MCB1700 Development Board Direction Right

Conclusion

This laboratory exercise provided a practical and effective introduction to embedded systems development using the Keil uVision IDE and the ARM Cortex M3-based MCB1700 board. Through the process of creating a joystick-controlled interface, fundamental skills in project configuration, C programming for microcontrollers, and hardware peripheral manipulation were acquired. The successful outcome, where joystick movements were accurately reflected on both an LCD and through LEDs, demonstrates a solid understanding of the concepts covered. This lab has built a strong foundation for tackling more complex embedded systems challenges in the future.